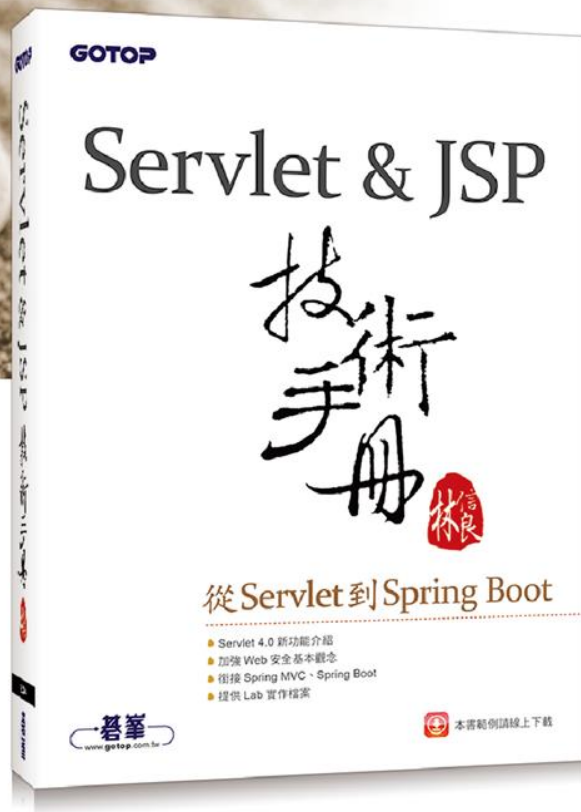




Servlet & JSP

技術手冊

從Servlet到Spring Boot

- Servlet 4.0 新功能介紹
- 加強 Web 安全基本觀念
- 銜接 Spring MVC、Spring Boot
- 提供 Lab 實作檔案

CHAPTER 1

• 簡介Web應用程式



學習目標

- 認識HTTP基本特性
- 了解GET、POST使用時機
- 了解何為URI編碼
- 認識Web容器角色
- 了解Servlet與JSP的關係
- 認識MVC/Model 2

Web應用程式基礎知識

- HTML (HyperText Markup Language)
- HTTP (HyperText Transfer Protocol)
- URI (Uniform Resource Identifier)
- 文字編碼
- 後端？前端？
- Web安全概念

Web應用程式

- 客戶端（Client）與伺服器端（Server）
- 瀏覽器（Browser）與HTTP伺服器
 - 瀏覽器請求伺服器上的檔案或資源
 - 對本書的主旨來說，伺服器的檔案或資源必須產生HTML。

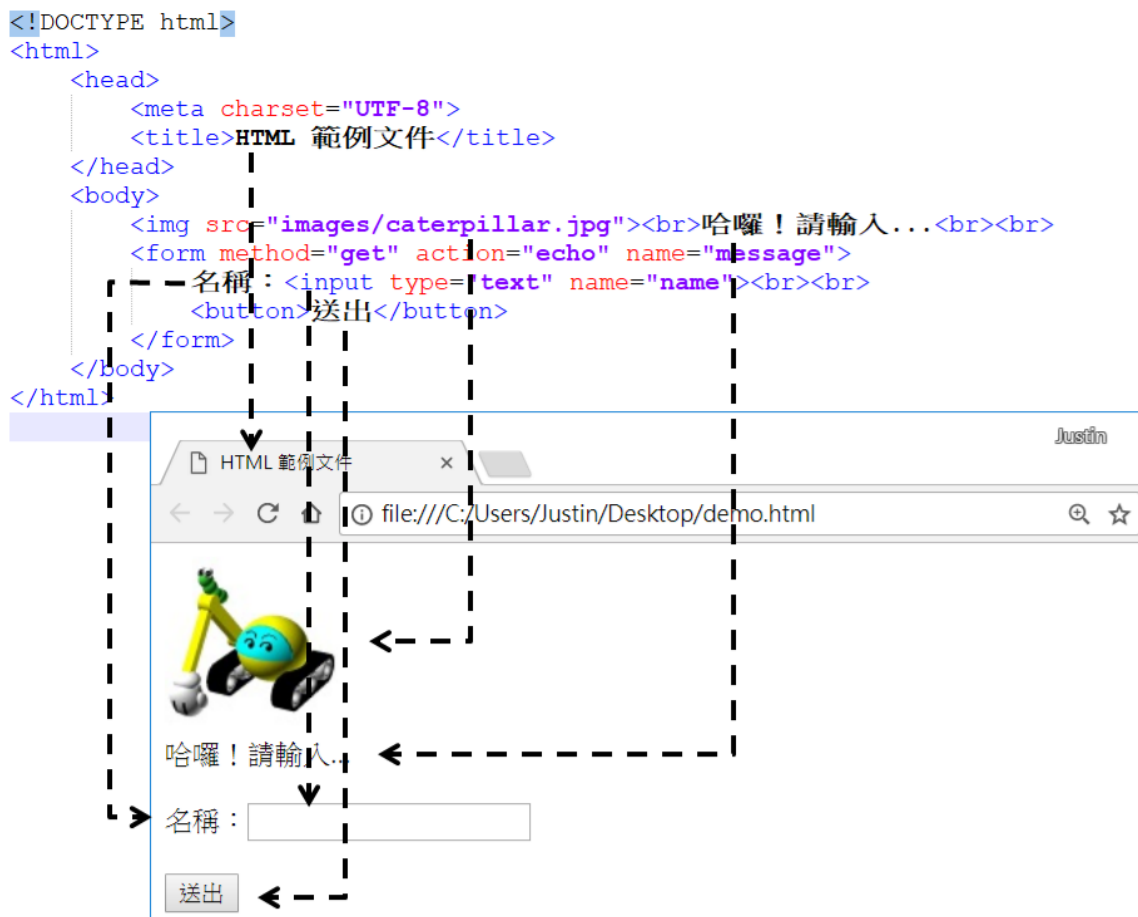
關於HTML

- 標籤（Tag）的方式來定義文件結構

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>HTML 範例文件</title>
  </head>
  <body>
    哈囉！請輸入...<br><br>
    <form method="get" action="echo" name="message">
      名稱:<input type="text" name="name"><br><br>
      <button>送出</button>
    </form>
  </body>
</html>
```

關於HTML

- 瀏覽器依HTML的結構等資訊進行畫面繪製



關於HTML

- w3schools
 - w3schools.com
- HTML 5 Tutorial
 - www.w3schools.com/html/

URL、URN與URI？

- URL：Uniform Resource Locator
- URN：Uniform Resource Name
- URI：Uniform Resource Identifier
- 早期U代表Universal（萬用）
- 標準化之後代表著Uniform（統一）

URL

- 主要格式

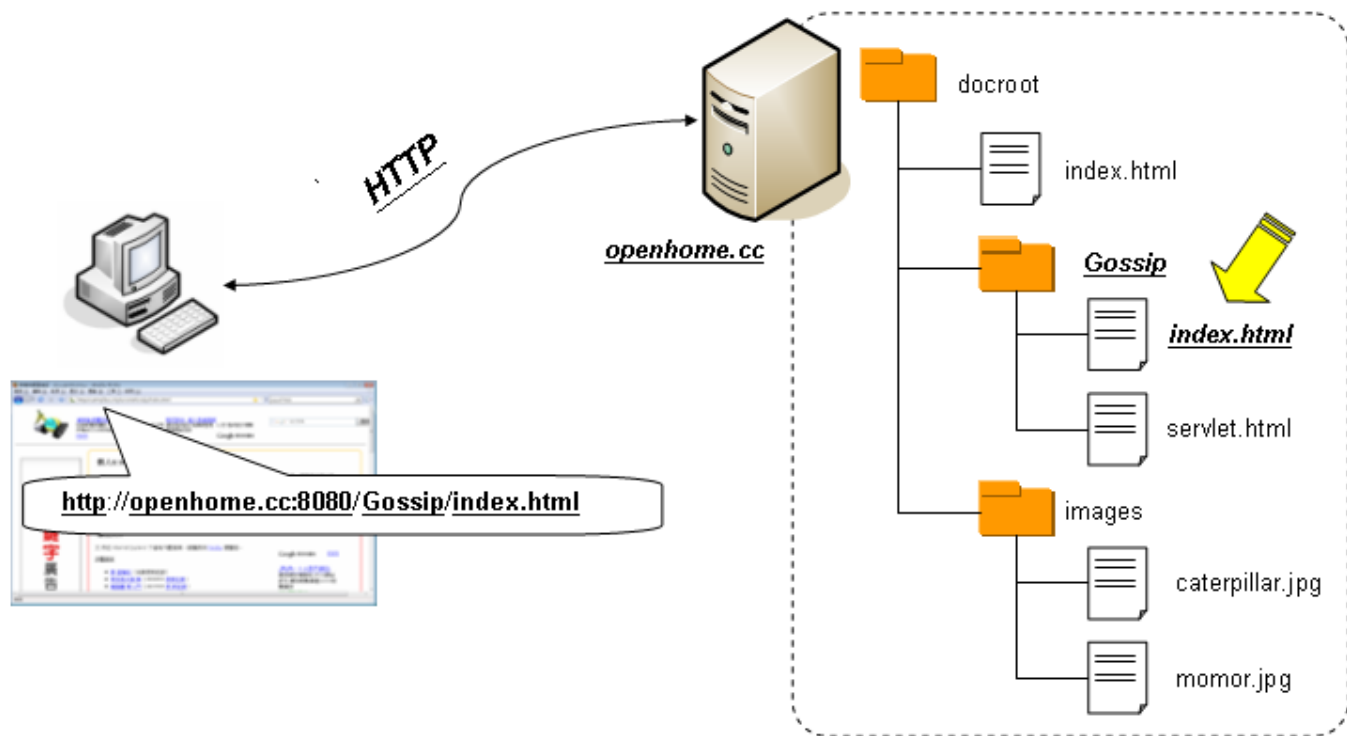
`<scheme>:<scheme-specific-part>`

- 協議（**scheme**）指定了以何種方式取得資源
 - ftp（檔案傳輸協定，File Transfer protocol）
 - http（超文件傳輸協定，Hypertext Transfer Protocol）
 - mailto（電子郵件）
 - file（特定主機檔案名稱）

URL

- 特定協議部份的格式

//<使用者>:<密碼>@<主機>:<埠號>/<路徑>



URL

- 檔案系統中C:\workspace下的jdbc.pdf檔案

`file:///C:/workspace/jdbc.pdf`

URN

- 代表某個資源獨一無二的名稱
- 《Java SE 9技術手冊》的國際標準書號：
– `urn:isbn:978-986-476-675-8`

URI

- URL與URN為URI的子集

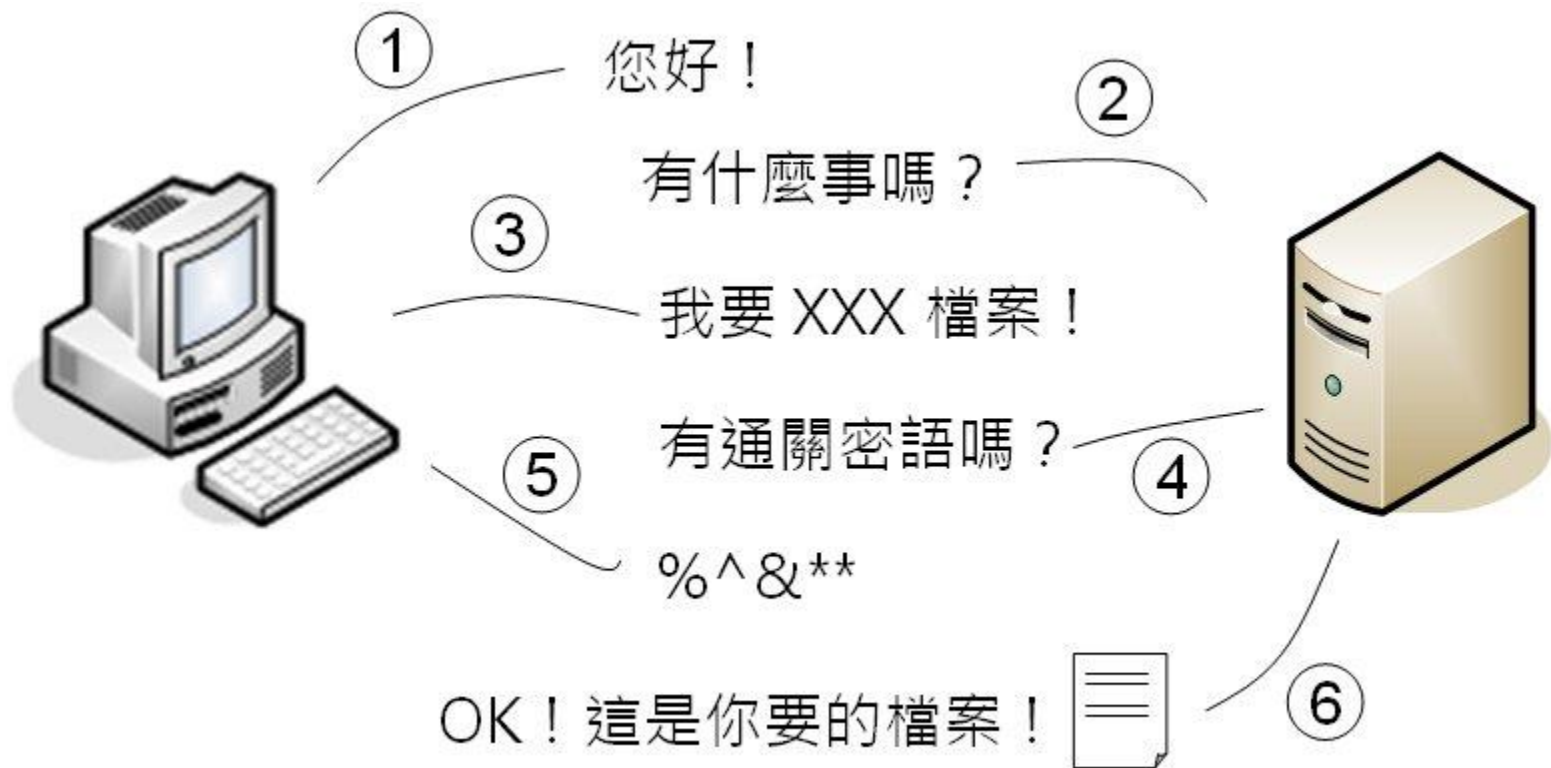
```
URI          = scheme ":" hier-part [ "?" query ] [ "#" fragment ]
hier-part    = "://" authority path-abempty
              / path-absolute
              / path-rootless
              / path-empty
```

```
foo://example.com:8042/over/there?name=ferret#nose
 \  /      \  /      / \  /      / \  /      / \  /
  |         |         |         |         |
scheme    authority  path      query    fragment
  |
  |         |         |         |         |
 / \      / \      / \      / \      / \
urn:example:animal:ferret:nose
```

URI

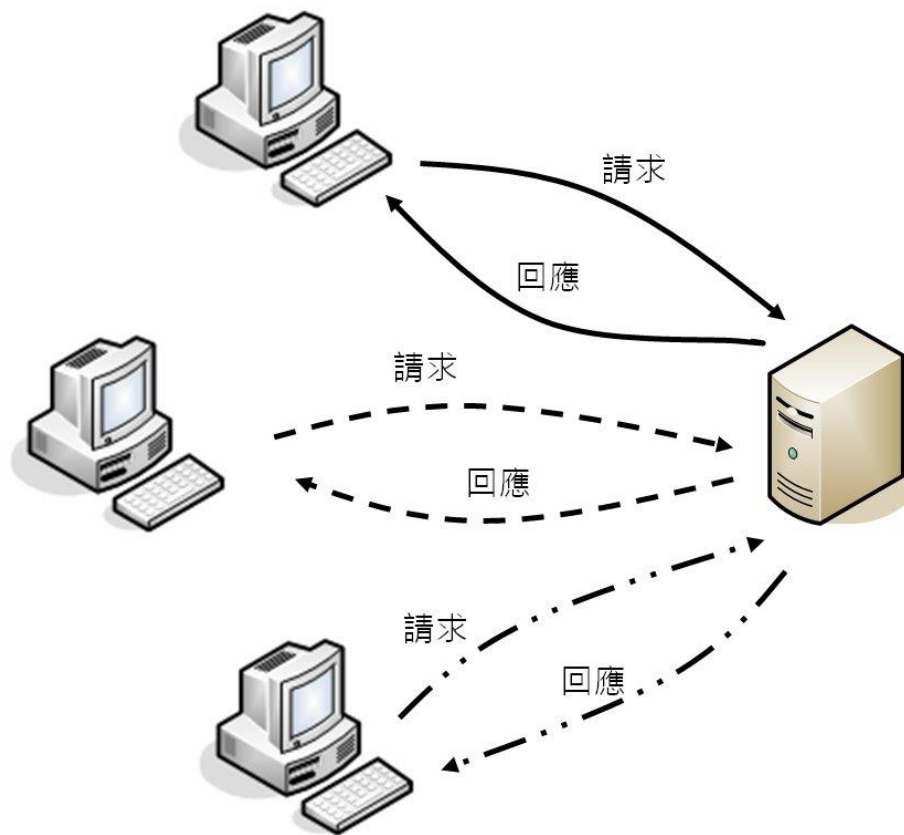
- 標準機構如W3C（World Wide Web Consortium）文件中，多使用URI
- 舊習慣使然，程式設計人員口語交談也多見使用 URL 這個舊稱
- 歷史演進與標準發佈
 - en.wikipedia.org/wiki/Uniform_Resource_Identifier

關於HTTP



關於HTTP

- 基於請求（Request）/回應（Response）模型
- 無狀態（Stateless）通訊協定



GET請求

- 向伺服器取得（GET）指定的資源

方法	路徑	請求參數	版本
GET	/Gossip/download?	file=servlet&user=caterpillar	HTTP/1.1
請求標頭			
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)			
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/63.0.3239.108			
Safari/537.36			
accept: text/xml,application/xml,image/gif,image/x-bitmap			
accept-encoding: gzip, deflate, br			
accept-language: zh-TW,zh;q=0.9,en-US;q=0.8,en;q=0.7,zh-			
CN;q=0.6,pt;q=0.5			
...			



<https://openhome.cc/Gossip/download?file=servlet&user=caterpillar>

GET請求

- 可以發送的請求參數長度有限（這個長度依瀏覽器版本而有所不同）
- 太大量資料並不適合用GET方法來進行請求

POST請求

- 請求時發佈（POST）資訊給伺服器



POST請求

- 大量資料的發送都會使用POST方法
- 請求參數移至訊息本體，網址列上也就不會出現請求參數
- 較敏感的資訊，即使長度不長，通常也會改用POST的方式發送

GET或POST ?

- 敏感資訊
- 書籤設置考量
- 瀏覽器快取
- 安全與等冪

GET或POST ?

- <form>預設會使用GET

...

```
<form method="get" action="download " name="filename">  
  名稱:<input type="text" name="name"><br><br>  
  <input type="button" value="送出">  
</form>
```

...

URI編碼

- 如果請求參數值本身包括=符號怎麼辦？

~~GET /Gossip/download?url=https://openhome.cc HTTP/1.1~~

- URI保留字元

— 「:」、「/」、「?」、「&」、「=」、「@」、「%」....

URI保留字元

- 要在請求參數上表達URI中的保留字元，必須在%字元之後以十六進位數值表示方式，來表示該字元的八個位元數值
 - 「:」 (00111010) → %3A
 - 「/」 (00101111) → %2F

```
GET /Gossip/download?url=https://openhome.cc HTTP/1.1
```



```
GET /Gossip/download?url=https%3A%2F%2Fopenhome.cc HTTP/1.1
```


URI編碼

- 使用`java.net.URLEncoder`類別的靜態`encode()`方法作編碼動作使用
- `java.net.URLDecoder`的靜態`decode()`方法作解碼動作

```
String text = URLEncoder.encode("http://openhome.cc ", "ISO-8859-1");
```

URI與HTTP編碼

- 在URI規範中，空白字元是編碼為%20
- 在HTTP規範中空白是編碼為「+」

中文字元

- URI規範的URI編碼，針對的是字元UTF-8編碼的八個位元數值
 - 林 (UTF-8) → %E6%9E%97
 - 伺服器端處理請求參數時，必須使用UTF-8編碼來取得正確的「林」字元

`https://openhome.cc/addBookmar.do?lastName=%E6%9E%97`

 安全 | `https://openhome.cc/register?lastName=林`

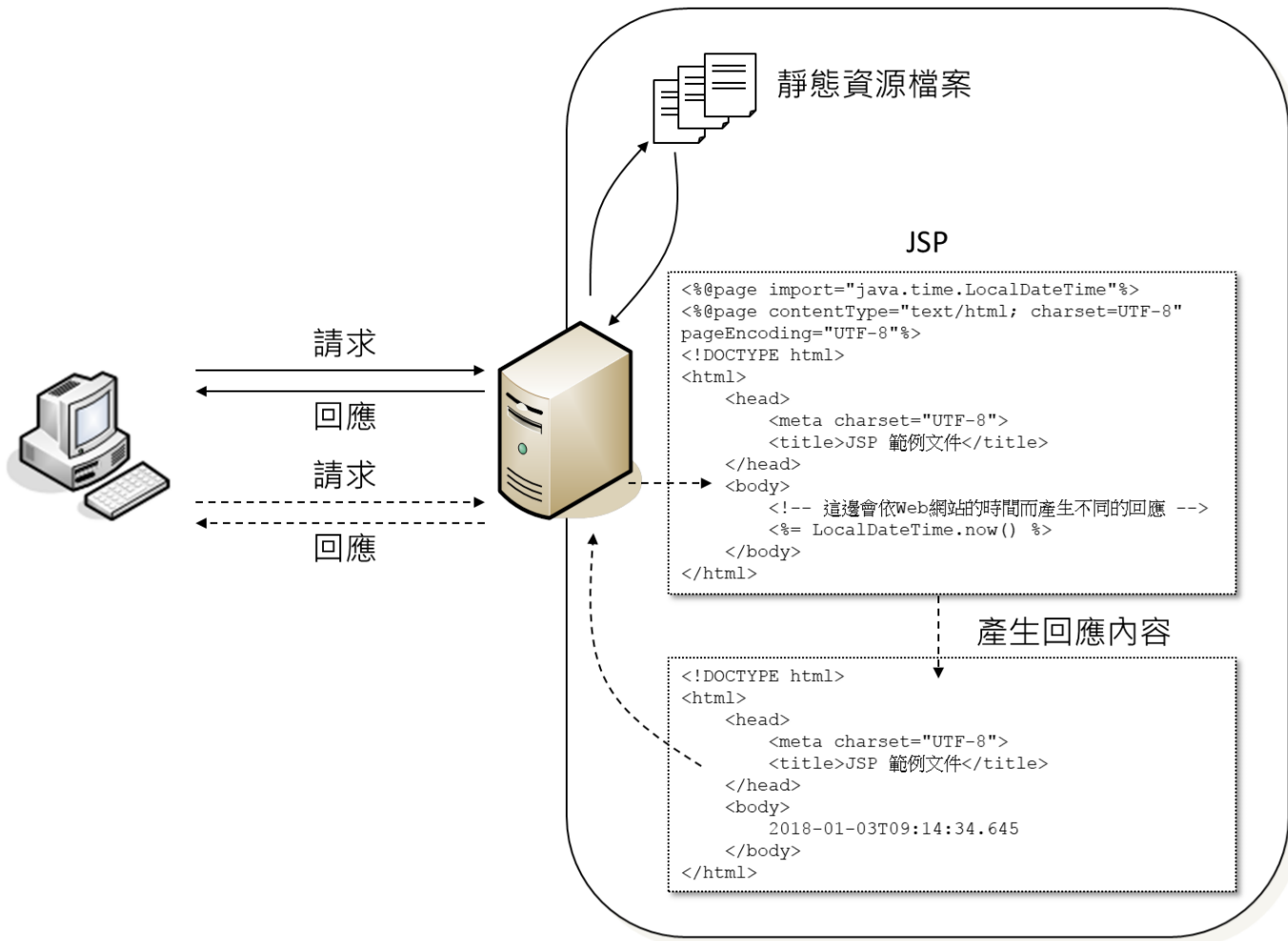
```
String text = URLEncoder.encode("林", "UTF-8");
```

中文字元

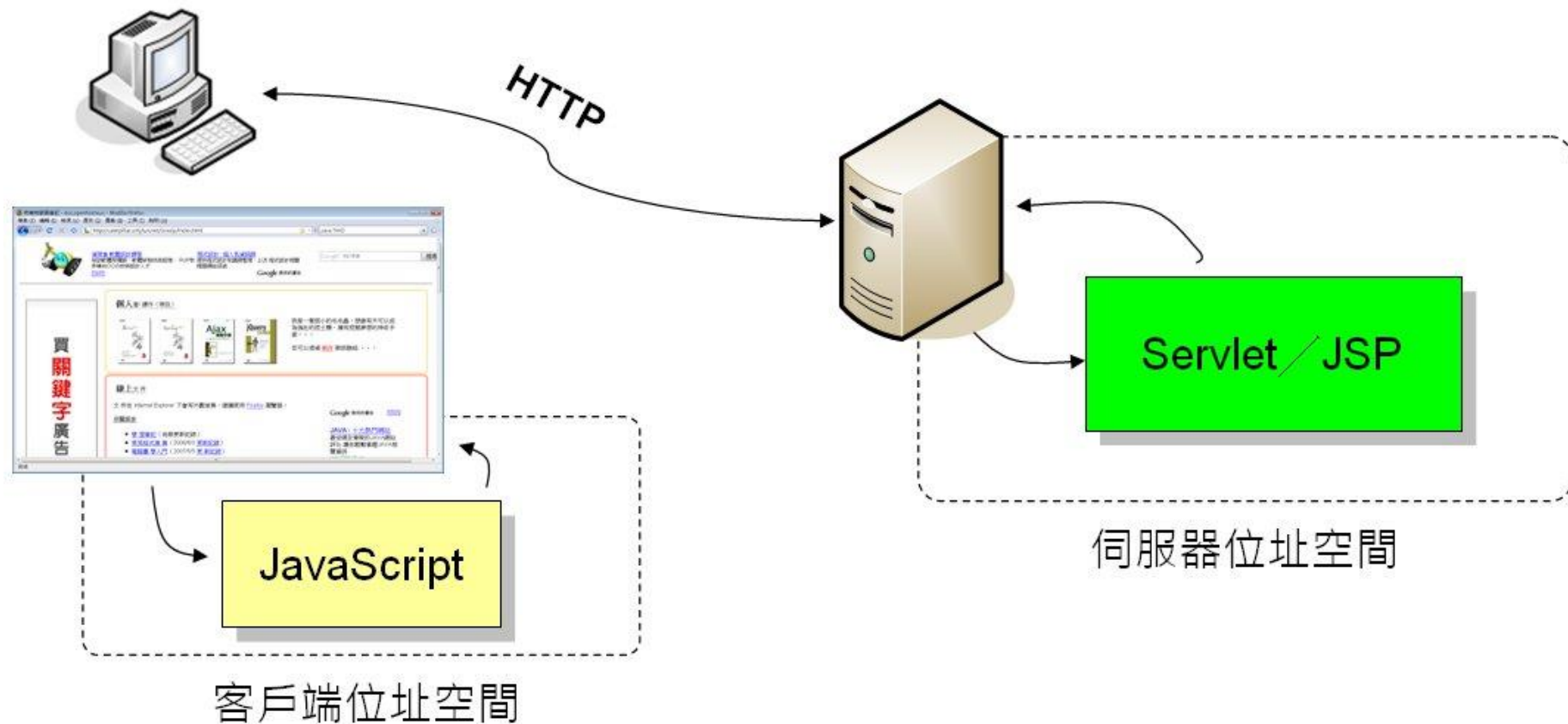
- 在一個MS950網頁中，若表單使用GET發送「林」這個中文字
 - 林 (MS950) → %AA%4C
 - 伺服器端處理請求參數時，就必須指定MS950編碼，以取得正確的「林」中文字元

```
String text = URLEncoder.encode("林", "MS950");
```

動態網頁？靜態網頁？



客戶端程式、伺服器端程式



Web安全觀念

- OWASP TOP 10
 - www.owasp.org/index.php/Category:OWASP_Top_Ten_Project
- 首次OWASP Top 10於2003發布
- 最新OWASP Top 10於2017年11月正式釋出

OWASP Top 10 - 2013	→	OWASP Top 10 - 2017
A1 – Injection	→	A1:2017-Injection
A2 – Broken Authentication and Session Management	→	A2:2017-Broken Authentication
A3 – Cross-Site Scripting (XSS)	↘	A3:2017-Sensitive Data Exposure
A4 – Insecure Direct Object References [Merged+A7]	U	A4:2017-XML External Entities (XXE) [NEW]
A5 – Security Misconfiguration	↘	A5:2017-Broken Access Control [Merged]
A6 – Sensitive Data Exposure	↗	A6:2017-Security Misconfiguration
A7 – Missing Function Level Access Contr [Merged+A4]	U	A7:2017-Cross-Site Scripting (XSS)
A8 – Cross-Site Request Forgery (CSRF)	⊗	A8:2017-Insecure Deserialization [NEW, Community]
A9 – Using Components with Known Vulnerabilities	→	A9:2017-Using Components with Known Vulnerabilities
A10 – Unvalidated Redirects and Forwards	⊗	A10:2017-Insufficient Logging&Monitoring [NEW,Comm.]

Web安全觀念

- CWE (Common Weakness Enumeration)
 - 始於2005年
 - 收集了近千個通用的軟體弱點
- CVE (Common Vulnerabilities and Exposures)
 - 就特定軟體發生的安全問題給予CVE-YYYY-NNNN形式的編號
 - 2017年底的CPU「推測執行」(Speculative execution) 安全漏洞
 - CVE-2017-5754、CVE-2017-5753與CVE-2017-5715

簡介Servlet/JSP

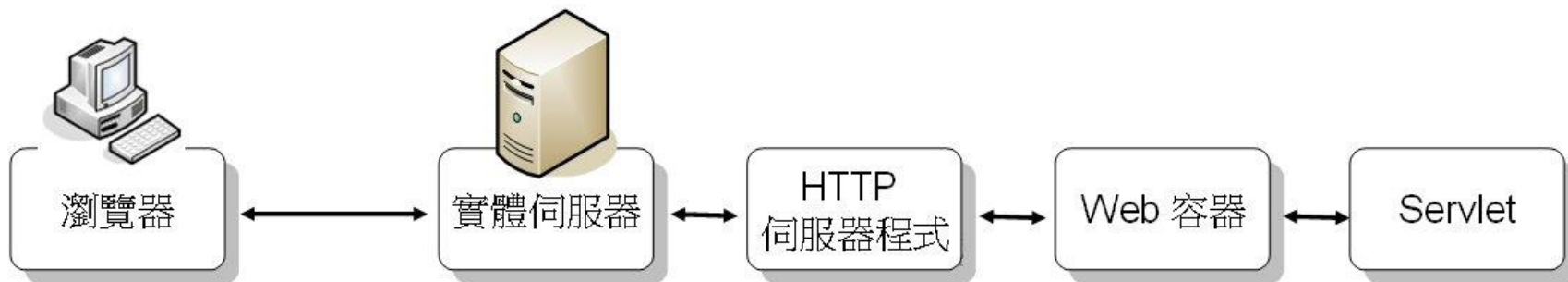
- JVM（Java Virtual Machine）是Java程式唯一認識的作業系統，其可執行檔為.class檔案
- Web容器（Container）是Servlet/JSP唯一認得的HTTP伺服器

何謂Web容器

- 對於撰寫Servlet/JSP來說，容器持有物件、負責物件生命周期與相關服務連結
- 具體層面....
 - 容器就是用Java寫的程式，運行於JVM之上
 - Servlet會接觸HttpServletRequest、HttpServletResponse等物件
 - HTTP文字性的通訊協定，如何變成Servlet/JSP中可用的Java物件？

何謂Web容器

- 抽象層面....
 - 可將Web容器視為運行Servlet/JSP的HTTP伺服器
- 就如同Java程式僅認得JVM這個作業系統，Servlet/JSP程式也僅認得Web容器這個概念上的HTTP伺服器



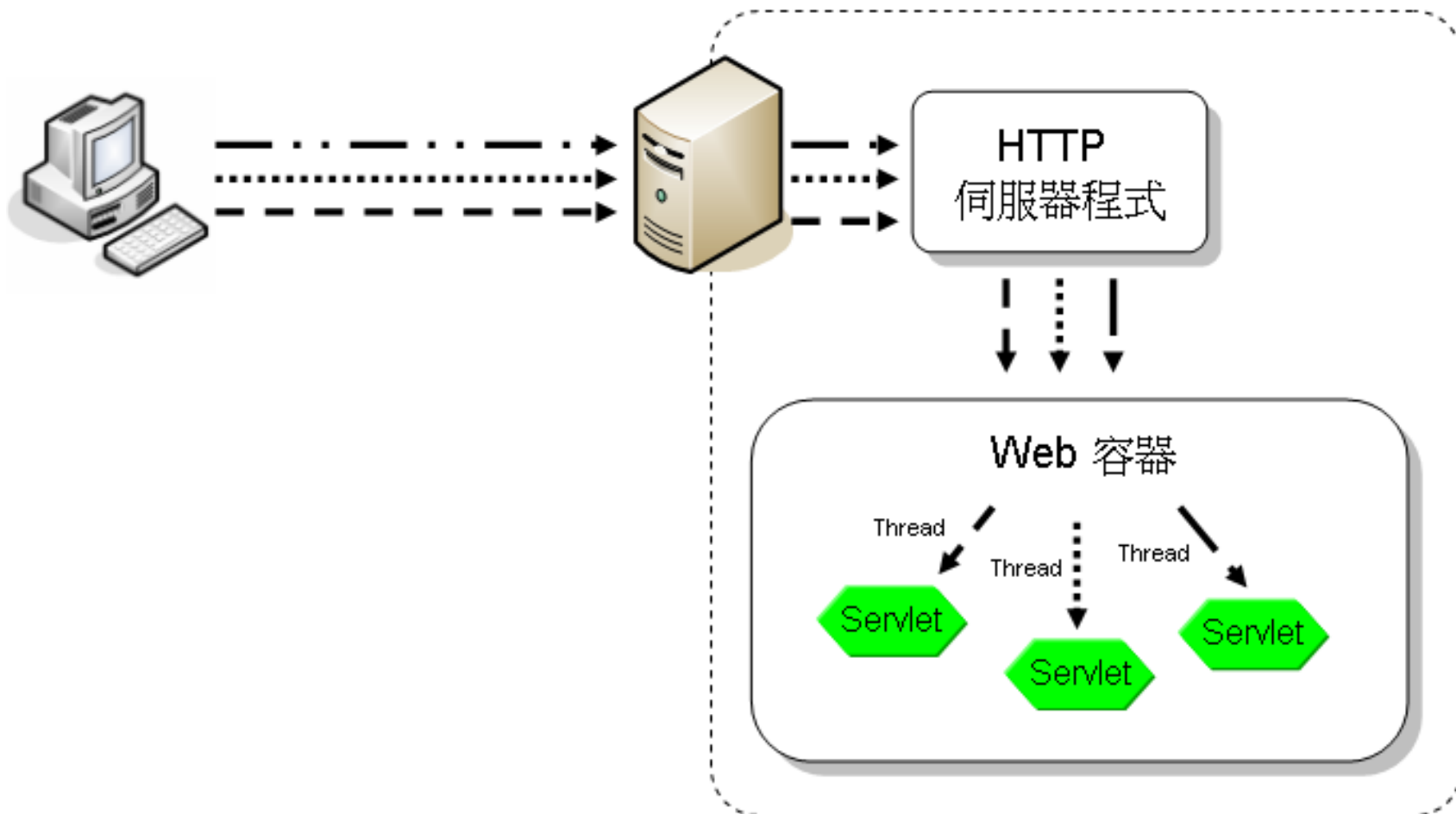
何謂Web容器

- JVM介於Java程式與實體作業系統之間
 - 撰寫Java程式必須了解 JVM與應用程式之間如何互動
- Web容器介於實體HTTP伺服器與Servlet之間
 - 撰寫Servlet/JSP也必須知道Web容器如何與Servlet/JSP互動，如何管理Servlet 等事實
 - JSP最後也是轉譯、編譯、載入為Servlet，在容器的世界中，真正負責請求、回應的是Servlet

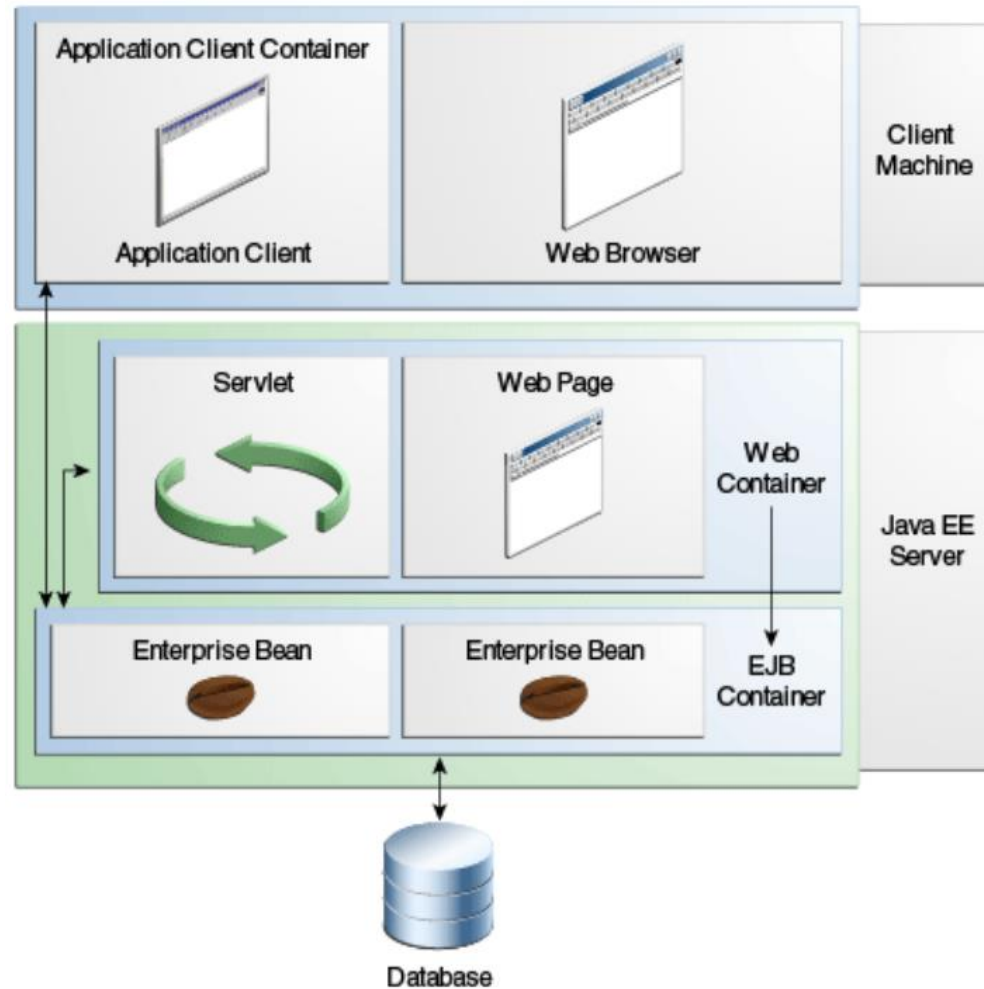
請求/回應範例

1. 客戶端（大部份情況下是瀏覽器）對Web伺服器發出HTTP請求。
2. HTTP伺服器收到HTTP請求，將請求轉由Web容器處理，Web容器會剖析HTTP請求內容，建立各種物件（像是HttpServletRequest、HttpServletResponse、HttpSession等）。
3. Web容器由請求的URI決定要使用哪個Servlet來處理請求（事先由開發人員定義）。
4. Servlet根據請求物件（HttpServletRequest）的資訊決定如何處理，透過回應物件（HttpServletResponse）來建立回應。

請求/回應範例



容器



Servlet與JSP

```
package cc.openhome;

import java.io.IOException;
import java.io.PrintWriter;
import java.time.LocalDateTime;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/time")
public class Time extends HttpServlet {
    @Override
    protected void doGet(
        HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        PrintWriter out = resp.getWriter();

        out.println("<!Doctype html>");
        out.println("<html>");
        out.println("<head>");
        out.println("<meta charset='UTF-8'>");
        out.println("<title>Servlet 範例文件</title>");
        out.println("</head>");
        out.println("<body>");
        out.println("</body>");
        out.println(LocalDateTime.now());
        out.println("</html>");
    }
}
```

Servlet與JSP

```
<%@page import="java.time.LocalDateTime"%>
<%@page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>JSP 範例文件</title>
  </head>
  <body>
    <!-- 這邊會依 Web 網站的時間而產生不同的回應 -->
    <%= LocalDateTime.now() %>
  </body>
</html>
```

```
package org.apache.jsp;

import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.jsp.*;
import java.time.LocalDateTime;

public final class time_jsp extends org.apache.jasper.runtime.HttpJspBase
    implements org.apache.jasper.runtime.JspSourceDependent,
               org.apache.jasper.runtime.JspSourceImports {

    // 略...

    public void _jspInit() {
    }

    public void _jspDestroy() {
    }

    public void _jspService(final javax.servlet.http.HttpServletRequest request,
        final javax.servlet.http.HttpServletResponse response)
        throws java.io.IOException, javax.servlet.ServletException {

        final java.lang.String _jspx_method = request.getMethod();
        if (!"GET".equals(_jspx_method) && !"POST".equals(_jspx_method)
        && !"HEAD".equals(_jspx_method)
        && !javax.servlet.DispatcherType.ERROR.equals(request.getDispatcherType())) {
            response.sendError(HttpServletResponse.SC_METHOD_NOT_ALLOWED, "JSPs only
            permit GET POST or HEAD");
            return;
        }
    }
}
```

```
// 略...
try {
    response.setContentType("text/html; charset=UTF-8");
    pageContext = _jspxFactory.getPageContext(this, request, response,
                                              null, true, 8192, true);
    // 略...
    out = pageContext.getOut();
    _jspx_out = out;

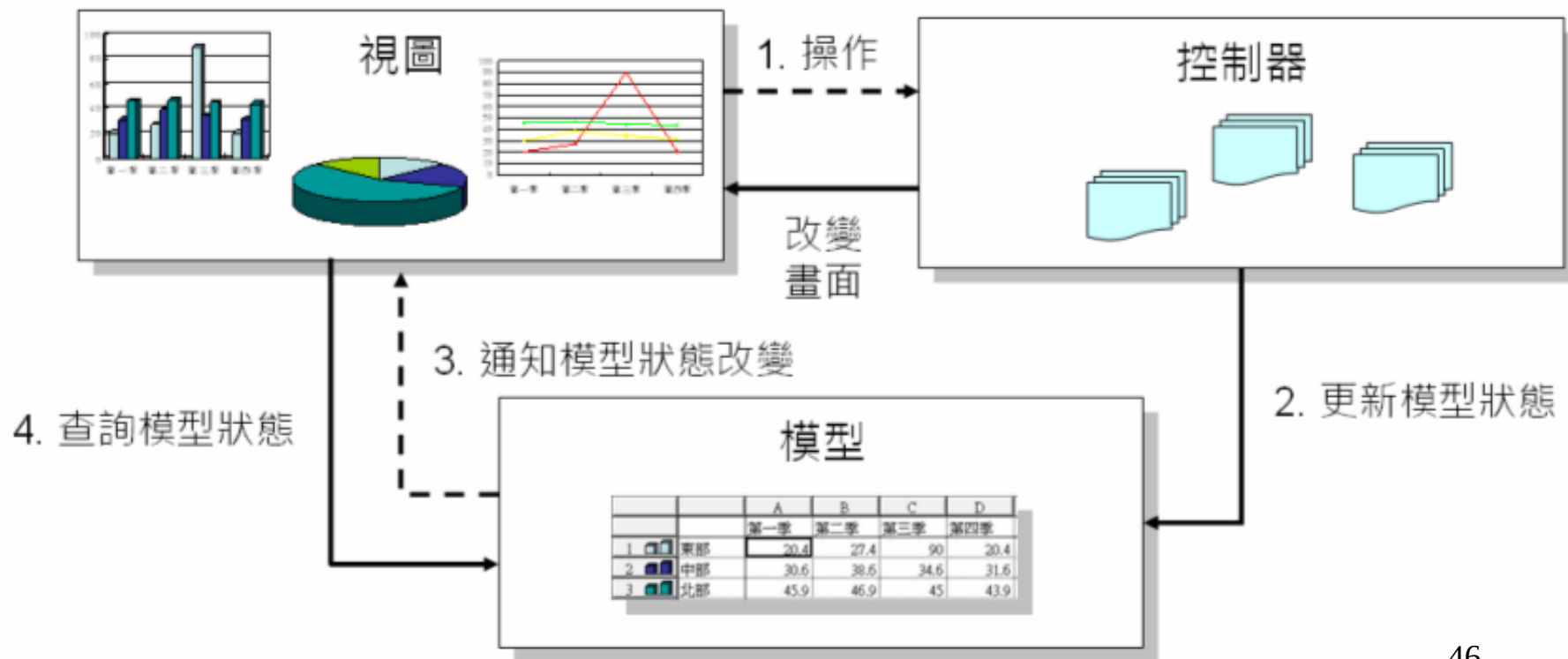
    out.write("\r\n");
    out.write("\r\n");
    out.write("<!Doctype html>\r\n");
    out.write("<html>\r\n");
    out.write("    <head>\r\n");
    out.write("        <meta charset=\"UTF-8\">\r\n");
    out.write("        <title>JSP 範例文件</title>\r\n");
    out.write("    </head>\r\n");
    out.write("    <body>\r\n");
    out.write("        ");
    out.print( LocalDateTime.now() );
    out.write("\r\n");
    out.write("    </body>\r\n");
    out.write("</html>");
} catch (java.lang.Throwable t) {
    // 略...
} finally {
    _jspxFactory.releasePageContext(_jspx_page_context);
}
}
```

關於MVC/Model 2

- 在Servlet程式中夾雜HTML的畫面輸出絕對不是什麼好主意
- 在JSP網頁中的HTML間夾雜Java程式碼，也是極度不建議的作法

MVC

- Model、View、Controller
- 模型、視圖、控制器



MVC

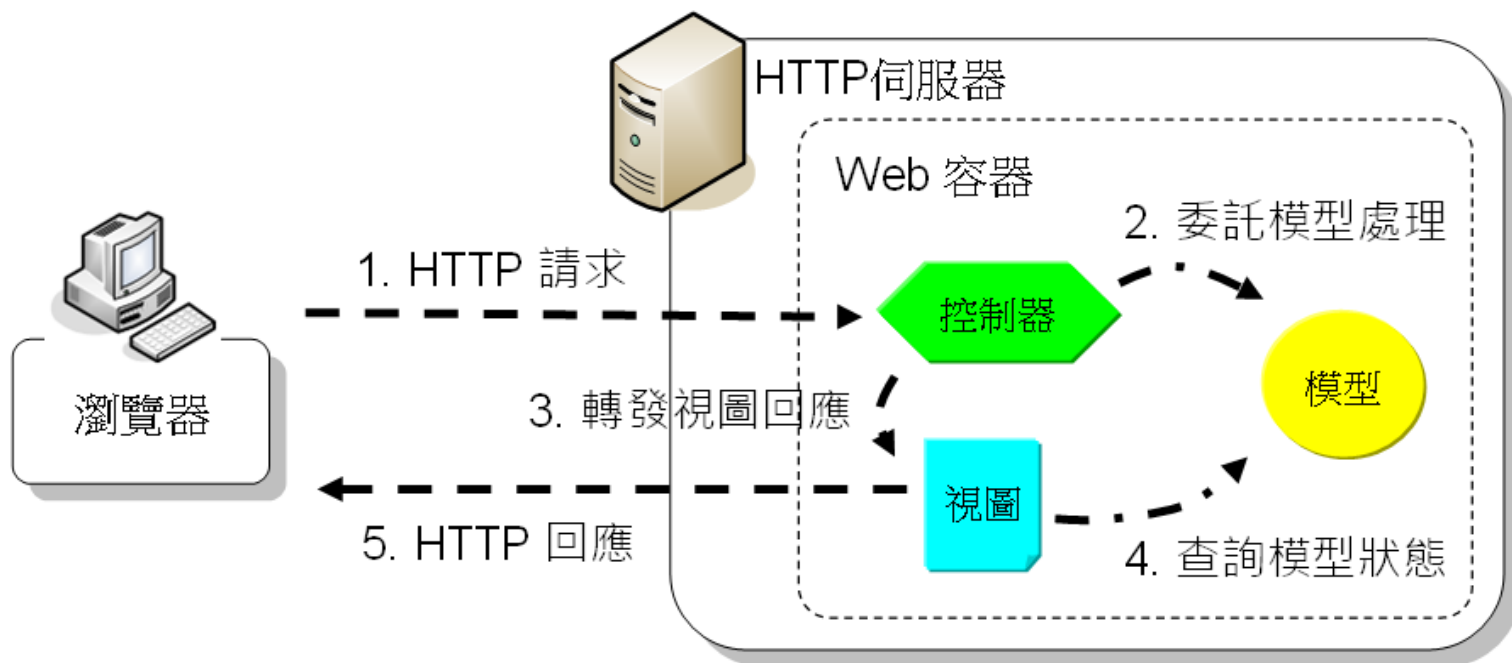
- 模型不會有畫面相關的程式碼
- 視圖負責畫面相關邏輯
- 控制器知道某個操作必須呼叫哪些模型

Model 2

- 套用在Web應用程式的設計上
 - 視圖部份可由網頁來實現
 - 伺服器上的資料存取或商務邏輯（Business logic）由模型負責
 - 控制器接送瀏覽器的請求，決定呼叫哪些模型來處理

Model 2

- Web應用程式是基於HTTP，必須基於請求/回應模型



Model 2

- 控制器（Controller）
 - 取得請求參數、驗證請求參數、轉發請求給模型、轉發請求給畫面，這些都使用程式碼來實現
- 模型（Model）
 - 接受控制器的請求呼叫，負責處理商務邏輯、負責資料存取邏輯等，這部份還可依應用程式功能，產生各多種不同職責的模型物件，模型使用程式碼來實現

Model 2

- 視圖（View）
 - 接受控制器的請求呼叫，會從模型提取運算後的結果，根據需求呈現所需的畫面，在職責分配良好的情況下，基本上可作到不出現程式碼，因此不會發生程式碼與HTML混雜在一起的情況

Model 2

```
<html>
<head>
  <meta content='text/html; charset=UTF-8' http-equiv='content-type'>
  <title>Gossip 微網誌</title>
  <link rel='stylesheet' href='css/member.css' type='text/css'>
</head>
<body>
  <div class='leftPanel'>
    <img src='images/caterpillar.jpg' alt='Gossip 微網誌' />
    <br><br>
    <a href='logout?username="${ sessionScope.login }'>登出 ${ sessionScope.login }</a>
  </div>
  <form method='post' action='message'>
    分享新鮮事...<br>
    <c:if test="${requestScope.blabla != null}">訊息要 140 字以內<br></c:if>
    <textarea cols='60' rows='4' name='blabla'>${requestScope.blabla}</textarea><br>
    <button type='submit'>送出</button>
  </form>
  <table style='text-align: left; width: 510px; height: 88px;'
    border='0' cellpadding='2' cellspacing='2'>
    <thead>
      <tr>
        <th><hr></th>
      </tr>
    </thead>
    <tbody>
      <c:forEach var="blah" items="${requestScope.blahs}">
        <tr>
          <td style='vertical-align: top;'>
            ${blah.username}<br>
            <c:out value="${blah.txt}" /><br>
            <fmt:formatDate value="${blah.date}" type="both"
              dateStyle="full" timeStyle="full"/>
            <a href='delete?message=${blah.date.time}'>刪除</a>
            <hr>
          </td>
        </tr>
      </c:forEach>
    </tbody>
  </table>
  <hr style='width: 100%; height: 1px;'>
</body>
</html>
```

Model 2

```
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org">
  <head>
    <meta content='text/html; charset=UTF-8' http-equiv='content-type'>
    <title>Gossip 微網誌</title>
    <link rel='stylesheet' href='css/member.css' type='text/css'>
  </head>
  <body>
    <div class='leftPanel'>
      <img src='images/caterpillar.jpg' alt='Gossip 微網誌' />
      <br><br>
      <a th:href="@{logout?username={username} (username=${session.login})}">登出</a>
    </div>
    <form method='post' action='message'>分享新鮮事...<br>
    <span th:if="${blabla != null}">訊息要 140 字以內</span><br>
      <textarea cols='60' rows='4' name='blabla' th:text="${blabla}">Blabla...</textarea><br>
      <button type='submit'>送出</button>
    </form>
    <table style='text-align: left; width: 510px; height: 88px;'
          border='0' cellpadding='2' cellspacing='2'>
      <thead>
        <tr>
          <th><hr></th>
        </tr>
      </thead>
      <tbody>
        <tr th:each="blah : ${blahs}">
          <td style='vertical-align: top;'>
            <span th:text="${blah.username}">user name</span><br>
            <span th:text="${blah.txt}">blabla</span><br>
            <span th:text="${#dates.format(blah.date, 'dd/MMM/yyyy HH:mm')}">time here</span>
            <a th:href="@{delete?message={time} (time=${blah.date.time})}">刪除 </a>
            <hr>
          </td>
        </tr>
      </tbody>
    </table>
    <hr style='width: 100%; height: 1px;'>
  </body>
</html>
```

簡介Java EE

- JCP、JSR
 - Java SE
 - Java ME
 - Java EE

簡介Java EE

- Java EE 8平台
 - www.oracle.com/technetwork/java/javaee/tech/index.html
- 主要規範是在JSR 366文件
 - Servlet 4.0規範在JSR 369
 - JSP 2.3規範在JSR 245
 - Expression Language 3.0規範在JSR341
 - JSTL 1.2規範於JSR52